pse-edge-mcp — Developer Walkthrough

Audience: a developer or architect who has never seen this codebase and needs to understand it, debug it, or extend it.

Companion documents. This file explains *how the system works*. Two others explain *why it is the way it is*, and you should read them before changing anything structural:

Document	Contents
`docs/plan.md`	Every design decision: scope, caching policy, auth design, phases, risks
`docs/endpoints.md`	The verified PSE Edge endpoint map — request dialects, parameter names, response shapes
`docs/deploy.md`	Running it in production
`CLAUDE.md`	The short form of the invariants, kept next to the code

Version described: **0.8.0**. 34 modules, ~7,000 lines of source, ~5,100 lines of tests.

---

1. What this is, in one paragraph

An **MCP server** (Model Context Protocol — a standard way to expose tools to an LLM client) that serves **Philippine Stock Exchange** data taken from the PSE Edge disclosure portal. It exposes 12 read-only tools: quotes, price history, disclosures, financial reports, dividends, and index/market data.

It is **unofficial**. PSE Edge publishes no API, so this server speaks to the same internal endpoints the portal's own web pages call.

The one decision that explains the whole design

PSE Edge is a public utility with no rate-limit contract and no commercial relationship with this project. Hammering it during trading hours would be antisocial and would get the project blocked. So:

> **This is an end-of-day server. It makes zero upstream requests while the market is open.**

Almost every structural choice follows from that sentence — the cache is a *freeze* rather than a TTL, every response carries freshness metadata, there is an opportunistic archive, and there is a politeness throttle independent of user quotas. If a change you are contemplating would increase load on PSE Edge, it is probably the wrong change.

---

2. Running it

Three modes, same tool implementations underneath.

```
MODE          COMMAND                                    AUTH   STORAGE
──────────────────────────────────────────────────────────────────────────
stdio         uvx pse-edge-mcp                           never  in-memory
(local)       (Claude Desktop / Claude Code default)

HTTP          pse-edge-mcp --http --port 8000            opt-in  in-memory or
(dev)                                                           Postgres

HTTP          docker compose -f compose.nas.yaml up -d   on     Postgres
(production)  + compose.tunnel.yaml
```

stdio never authenticates. It runs on the user's own machine as a subprocess of their client; there is no network boundary to guard and no second party to authenticate. Adding auth there would be pure friction.

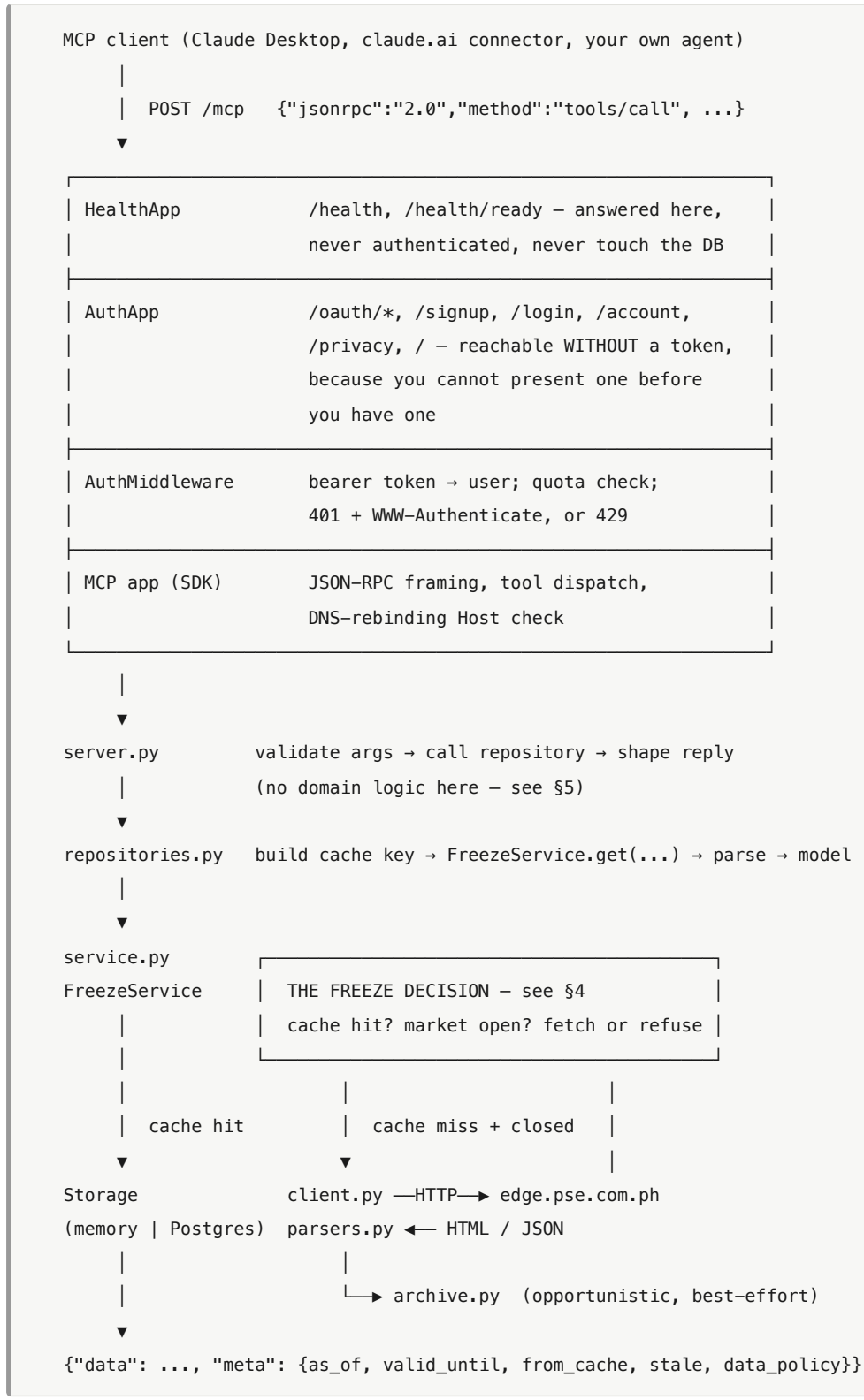
Local development:

```
uv sync --all-extras          # Python 3.14, uv for everything
uv run pytest                 # 266 tests, no network access
uv run ruff check .           # line length 100
uv run mypy src               # strict
```

---

3. Request lifecycle

This is the path every tool call takes. Read it once and most of the codebase becomes predictable.



Two things worth noticing:

- **The composition is built exactly once**, in `asgi.py`. The CLI (`__main__.py`) and production both call `create_app()`, so they cannot drift apart.
- **`/mcp` is the only authenticated path.** Everything the browser needs during signup is deliberately in front of the auth gate.

---

4. The freeze policy — the core behaviour

This is the part to understand before anything else, because it is what makes the server *correct* rather than merely functional.

`FreezeService.get()` in `service.py` implements one decision table:

	MARKET CLOSED	MARKET OPEN
cache fresh	serve from cache	serve from cache
cache stale (expired)	FETCH upstream, store, serve	serve the stale value, flagged meta.stale = true ← never fetches
cache empty	FETCH upstream, store, serve	raise MARKET_OPEN_NO_CACHE with a retry_after timestamp

Market hours are **09:30–15:00 Asia/Manila on trading days**; weekends and the `PSE_HOLIDAYS` table in `market_calendar.py` are non-trading.

Consequences a newcomer will trip over

- **`MARKET_OPEN_NO_CACHE` is not a bug.** It is the policy working. Agents must handle it; a `retry_after` timestamp is included so they can schedule rather than poll.
- **`meta.stale = true` means the market is open** and you are seeing the last close. It does not mean the data is wrong.
- **Concurrent cache misses collapse into one upstream request** (`SingleFlight` in `ratelimit.py`). Ten simultaneous first-time requests for the same symbol produce one HTTP call, not ten.
- **`immutable=True`** (passed by the disclosure-detail path) marks objects that never change upstream — a disclosure identified by `edge_no`. Those are cached forever with `valid_until: null` and `data_policy: "immutable"`. The open-market freeze still governs their *first* fetch: protecting PSE Edge outranks serving a cache miss promptly.

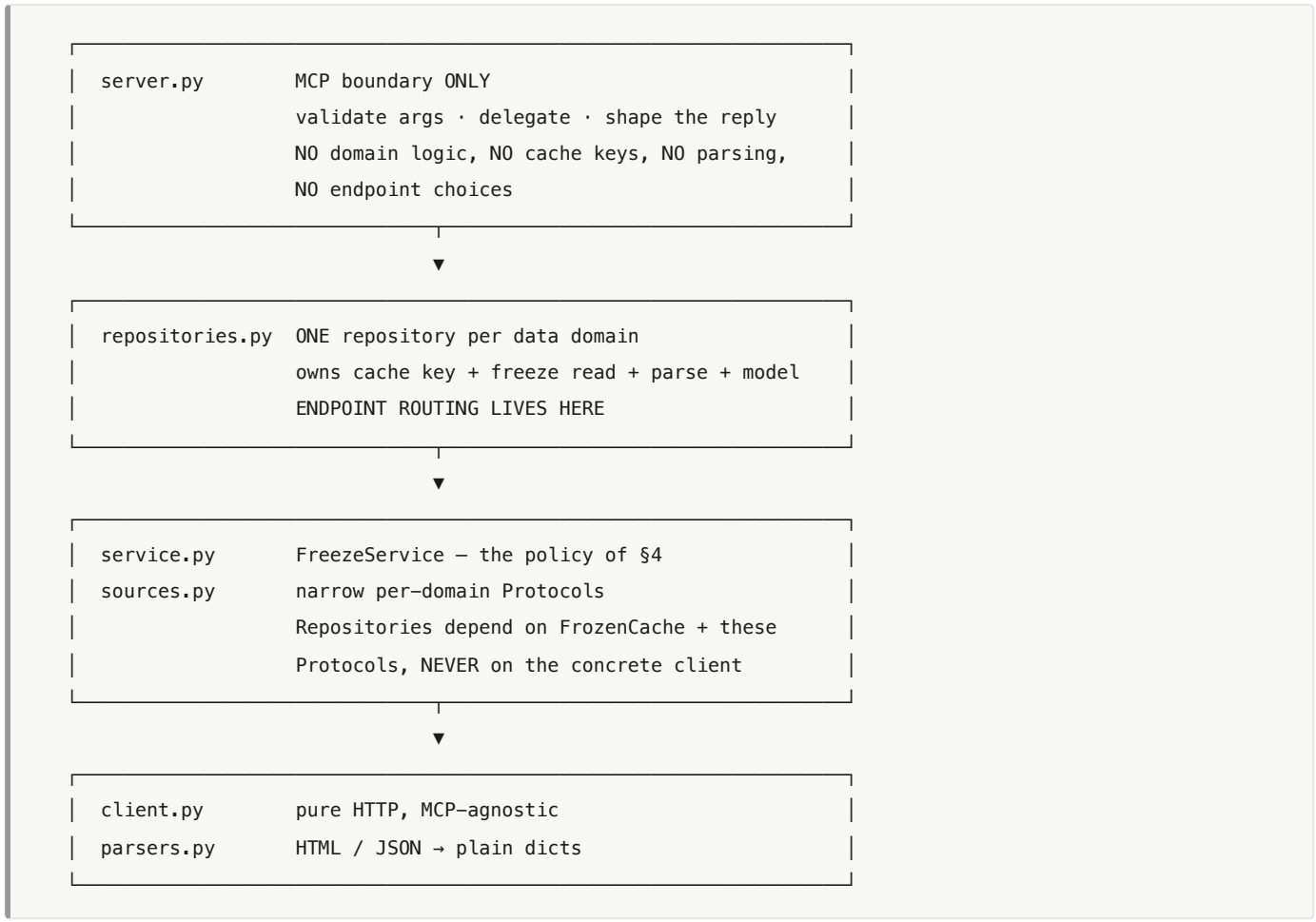
The politeness throttle is separate from quotas

`throttle_rate_per_sec` (default 1.0/s, burst 2) limits **outbound** requests to PSE Edge and exists to protect *them*. Per-user quotas (60/min, 2000/day) limit **inbound** requests and exist to protect *you*. Do not conflate them; raising one does not justify raising the other.

---

5. Architecture and layering

The layering is enforced by convention and by tests. Violating it is the most likely way a well-meaning change gets rejected in review.



Supporting modules: `validation.py` (argument validators), `models.py` (Pydantic), `errors.py`, `cache.py` (`Storage` protocol), `archive.py`, `ratelimit.py`, `market_calendar.py`, `config.py`, `memo.py`.

Why repositories depend on Protocols

`sources.py` defines five narrow interfaces — `CompanySource`, `QuoteSource`, `DisclosureSource`, `CompanyInfoSource`, `MarketSource`. `PseEdgeClient` happens to satisfy all of them, but repositories only ever see the narrow one they need. The payoff is in `tests/test_repositories.py`: a repository can be tested with a five-line fake and **no HTTP mocking at all**.

Module map

Module	Lines	Responsibility
`parsers.py`	752	HTML/JSON → dicts. The most PSE-specific code in the repo
`auth_app.py`	669	Browser-facing routes: OAuth, signup, login, account, privacy
`repositories.py`	511	The domain layer — five repositories
`passkeys.py`	424	WebAuthn ceremonies and browser sessions
`oauth.py`	417	OAuth 2.1 server: DCR, PKCE, code exchange, refresh rotation
`server.py`	412	The 12 tool definitions
`admin.py`	323	`pse-edge-admin` CLI
`models.py`	318	23 Pydantic models
`client.py`	268	HTTP to PSE Edge; both request dialects
`db.py`	241	SQLAlchemy tables, engine, schema check
`asgi.py`	217	The single composition point
`auth.py`	190	`TokenService`, `QuotaTracker`. Must stay SQLAlchemy-free

---

6. The tool surface

All 12 tools are read-only and return the same envelope (§7).

Tool	Arguments	Repository
`search_companies`	`query`	`CompanyRepository.search`
`validate_symbol`	`symbol`	`CompanyRepository.try_resolve`
`get_stock_quote`	`symbol`	`QuoteRepository.quote`
`get_price_history`	`symbol`, `start_date?`, `end_date?`	`QuoteRepository.history`
`search_disclosures`	`symbol?`, `start_date?`, `end_date?`, `template?`, `page`	`DisclosureRepository.search`
`search_disclosure_fulltext`	`keyword`, `start_date?`, `end_date?`, `symbol?`, `subject_title?`, `page`	`DisclosureRepository.fulltext`
`get_disclosure`	`edge_no`	`DisclosureRepository.detail`
`get_company_profile`	`symbol`	`CompanyInfoRepository.profile`
`get_financial_highlights`	`symbol`	`CompanyInfoRepository.financials`
`get_dividends_and_rights`	`symbol`	`CompanyInfoRepository.dividends_and_rights`
`get_indices`	—	`MarketRepository.indices`
`get_market_summary`	—	`MarketRepository.summary`

Dates are `YYYY-MM-DD` on the tool surface; the client converts to PSE Edge's `MM-dd-yyyy` wire format.

`search_disclosures` routes to two different upstream endpoints depending on whether `symbol` is supplied — market-wide with a date range, or one company's full history. That routing decision lives in the repository, which is exactly where the layering says it belongs.

`search_disclosure_fulltext` is deliberately a separate tool, not a better search. PSE Edge's keyword index only covers roughly 2023–2025, so it is not a superset of `search_disclosures`. The tool reports this coverage limit in its own results.

---

7. The data contract

Every tool returns this envelope

```
{
  "data": { "...": "the payload" },
  "meta": {
    "as_of":       "2026-07-31T15:00:00+08:00",
    "valid_until": "2026-08-01T15:00:00+08:00",
    "from_cache":  true,
    "stale":       false,
    "data_policy": "EOD-frozen"
  }
}
```

`meta` is not decoration. It is the mechanism by which the server is honest about a freeze policy that would otherwise look like stale data. A client that ignores `meta` will eventually mislead someone about when a price was true.

- `data_policy: "immutable"` with `valid_until: null` marks objects that never change upstream.
- **Clients should cache against `valid_until`.** Re-querying before it passes returns a byte-identical answer and only burns quota.

Error codes

Errors are returned as data — `{"error": "CODE", "message": ..., ...}` — not as exceptions, so an LLM can react to them rather than seeing a traceback.

Code	Meaning	Caller should
`MARKET_OPEN_NO_CACHE`	Nothing cached and the market is open	Retry after `retry_after`
`SYMBOL_NOT_FOUND`	No such ticker	Try `search_companies`
`INVALID_ARGUMENT`	Failed validation	Fix the arguments
`ENDPOINT_CHANGED`	PSE Edge's response shape changed	**Alert a human — see §11**
`EDGE_UNAVAILABLE`	Upstream unreachable or erroring	Retry later
`INTERNAL_ERROR`	Anything else	File a bug

The mapping happens once, in `server.reply()`. Do **not** add per-tool `try/except`.

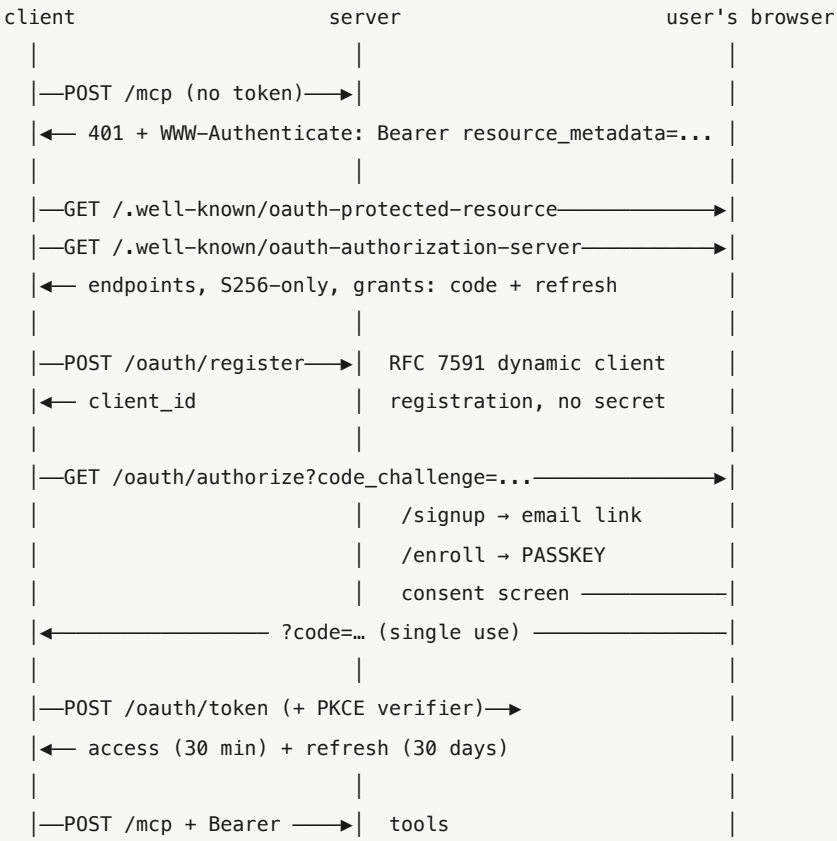
> **`reply()` is a helper that takes a callable, and deliberately not a decorator.** The MCP SDK builds each tool's output schema from its return annotation, and a `functools.wraps` wrapper makes `inspect.signature` follow `__wrapped__` to the *inner* annotation, breaking schema generation. Tools must stay annotated `-> dict[str, Any]`. `tests/test_server_disclosures.py::test_tool_surface_is_stable` guards this.

---

8. Authentication

Auth is **opt-in** (`PSE_AUTH_REQUIRED=1`, which requires `DATABASE_URL`). stdio never authenticates.

The self-service flow



Exactly two steps involve a human: the passkey enrollment and the consent click. No passwords exist anywhere in the system.

Rules that must not regress

- Unknown client or unregistered redirect URI is **fatal and never redirects** — otherwise the server is an open redirector.
- Redirect URIs match **exactly**. No prefix matching.
- **PKCE is mandatory, S256 only.**
- Authorization codes are single-use via one atomic `UPDATE … WHERE consumed_at IS NULL RETURNING`.
- Refresh tokens rotate; **replaying a rotated one revokes the entire family** (RFC 9700 §4.14). A stolen refresh token gets one use before the theft is detected.
- Only `kind='access'` authenticates as a bearer.
- Tokens are opaque, `pse_`-prefixed, stored **only as SHA-256**.

- The validation-cache TTL (default 60 s) is **the revocation-latency budget and nothing else**. Refusals are never cached.
- `auth.py` must stay **SQLAlchemy-free** so the lean install path works; Postgres code lives in `auth_store.py` and imports lazily.

Headless and agentic access

There is **no `client_credentials` grant** — both supported grants are browser-bound. A headless agent therefore uses an operator-issued token:

```
pse-edge-admin create-user agent@example.com
pse-edge-admin issue-token agent@example.com --note nightly-job   # shown once
```

Give each agent its **own user**: quotas are per user, so a runaway job throttles itself, and revocation is surgical. This is also the only route on a plain-HTTP deployment, since browsers restrict WebAuthn to secure contexts.

Privacy surface

`/account` shows a signed-in user everything held about them; `POST /account/delete` erases it immediately, in one transaction, with no approval step. Usage is aggregated **per user-hour, never per request** — minimal collection *and* no write on the hot path — and purged after 90 days.

> `tests/test_privacy.py::test_erasure_leaves_nothing_behind` walks `metadata.tables`, so a new user-keyed table **fails that test** rather than silently retaining personal data. If you add a table with a user foreign key, that test is your reminder to wire it into erasure.

---

9. Storage and the archive

Postgres is **optional**. `DATABASE_URL` unset gives an in-memory cache and a `NullArchive` — the zero-config stdio path, which does not even need the `postgres` extra installed.

```
DATABASE_URL unset          DATABASE_URL set
____________________        ________________________________________

InMemoryStorage             PostgresStorage  → shared cache, so
NullArchive                   N replicas still cause exactly one
(per-process, ephemeral)      upstream fetch per boundary
                            PostgresArchive → EOD bars and
                              disclosures accumulate over time
```

Tables: `cache_entries`, `eod_bars`, `disclosures`, `users`, `auth_tokens`, `webauthn_credentials`, `web_sessions`, `oauth_clients`, `oauth_flows`, `email_verifications`, `usage_events`.

The archive is opportunistic. Nothing crawls. It fills only from fetches a user already caused, so it deepens over time at zero additional cost to PSE Edge — which matters because PSE Edge itself serves only limited history.

Rules:

- **Schema is applied by Alembic only**, never `create_all` at runtime. Compose runs a one-shot `migrate` service and `check_schema()` fails loudly at startup if migrations were skipped.
- **A failed archive write must never fail a read.** It is caught broadly — a dead database raises `OSError`, not merely `SQLAlchemyError`.

- Postgres imports are **lazy**, inside `build_storage()`. A test asserts SQLAlchemy does not leak into `sys.modules` on `import pse_edge_mcp.server`.

---

10. Extending the server

Adding a tool to an existing domain

1. Add a method to the relevant repository in `repositories.py`. It owns the cache key, the `FreezeService.get()` call, the parse, and the model.
2. Add a thin tool in `server.py`: validate arguments, call the repository, `return await reply(run)`. Keep the `-> dict[str, Any]` annotation.
3. Add a Pydantic model in `models.py` if the shape is new.
4. Record a fixture in `tests/fixtures/` and test the repository with a fake source.

Adding a whole data domain

A new domain is **a new repository plus thin tools** — never fetch/parse orchestration in `server.py`. Concretely:

1. Add the narrow Protocol to `sources.py`.
2. Add the fetch method to `client.py`, using the correct dialect (§11).
3. Add the parser to `parsers.py`. Parse by **`<thead>` label, never column position.**
4. Add the repository, the models, and the tools.
5. Capture a real fixture; tests never touch the network.

Checklist before opening a PR

- `uv run ruff check .` · `uv run mypy src` · `uv run pytest` all clean
- New endpoint → new fixture, and `docs/endpoints.md` updated
- Shape drift raises `EndpointChangedError` rather than returning partial data
- All reads go through `FreezeService.get()` — **never call `PseEdgeClient` from a tool**
- Bump `version` in `pyproject.toml` and roll `CHANGELOG.md`'s Unreleased section **in the same PR** as the work being released

Branching

All work lands on `develop`. `develop` reaches `main` only by pull request; `main` is protected. **Merging to `main` publishes a multi-arch image**, and a merge that changes `version` also cuts a GitHub Release and an immutable `:<version>` tag. Never commit to `main` directly.

---

11. Debugging guide

Symptom → cause

Symptom	Almost certainly
`POST /mcp` → **421 Invalid Host header**	The SDK's DNS-rebinding guard. `PSE_PUBLIC_URL` must match the host clients actually use — `asgi.transport_security_for()` derives the allowlist from it
OAuth completes, then the **first tool call fails**	Same as above. Everything about auth is fine; look at the transport layer
`MARKET_OPEN_NO_CACHE`	Working as designed. Retry after the close
`ENDPOINT_CHANGED`	PSE Edge changed shape. **Re-capture the fixture, compare against `docs/endpoints.md`, fix the parser.** Never paper over it
HTTP **415** from an `.ax` endpoint	Wrong dialect — that endpoint needs a JSON body, not form encoding
Disclosure results in the wrong order	`sortType` must be the literal string `"date"`
Signup → **503**	The mail provider refused. The log names the sender address; ZeptoMail verifies **exact** domains, so a verified `example.com` does not cover `sub.example.com`
Passkeys "just don't work"	`PSE_PUBLIC_URL` does not match the browser's origin. Enrolled credentials cannot be migrated, only re-enrolled
App container stopped, **empty log**	A start-time failure, not a crash — a crash always leaves logs. Usually a taken host port, or a bind-mount whose source does not exist
NAS UI shows the project as **"Error"**	`migrate` is a one-shot that exits 0. That is success. Judge health by `curl /health`
`ImportError` on sqlalchemy in a lean install	Something imported Postgres code eagerly. It must be lazy, inside `build_storage()`

Useful commands

```
curl -s https://<host>/health                                  # liveness
curl -s https://<host>/health/ready                            # readiness (checks DB)
curl -s https://<host>/.well-known/oauth-protected-resource    # is PSE_PUBLIC_URL right?
curl -sS -o /dev/null -w '%{http_code}\n' -X POST https://<host>/mcp   # expect 401

docker compose -f compose.nas.yaml logs -f app
docker compose -f compose.nas.yaml exec app pse-edge-admin list-users
```

`PSE_LOG_JSON=1` gives one JSON object per log line. Secrets are redacted as a backstop — nothing in this codebase logs a credential deliberately.

Two PSE Edge dialects

Getting this wrong produces an HTTP 415 that looks like a server fault:

```
JSON dialect        POST  /common/DisclosureCht.ax
                    Content-Type: application/json
                    → JSON body.   Form encoding gives 415.

Form dialect        POST  /announcements/search.ax
                    POST  /companyDisclosures/search.ax
                    POST  /keyword/search.ax
                    Content-Type: application/x-www-form-urlencoded
                    → an HTML FRAGMENT, not JSON.

Wire dates are MM-dd-yyyy.  PSE Edge's parameter names sometimes
differ from its own HTML form field names — trust docs/endpoints.md
over the page source.
```

---

12. Testing

Tests never touch PSE Edge. All HTTP is mocked with `respx` against 15 fixtures in `tests/fixtures/`, recorded from real captures. A new endpoint requires a new fixture.

Layer	How it is tested
Repositories	Five-line fake sources — no HTTP mocking (`test_repositories.py`)
Client / parsers	`respx` + recorded fixtures
Freeze policy	Injected calendar pinning the clock (`test_service.py`)
Postgres	testcontainers, marked `@pytest.mark.postgres`
Auth	`test_auth_journey.py` — soft-webauthn, **real signatures**, real Postgres
Deployment	`test_deploy.py` — health probes, logging, the real ASGI factory

A lesson worth internalizing

250 passing tests once missed a bug that made the server unusable in production: the transport-security setting. The journey test passed `transport_security=...` **explicitly**, so it exercised a configuration production never built — the fixture worked around the exact defect it should have caught.

> **A fixture that configures something production leaves at its default is a blind spot, not a test.** Where practical, test through `create_app()` — the real factory — rather than hand-assembling the stack.

---

13. Configuration reference

All settings live in `config.py` as a frozen dataclass; environment variables override.

Variable	Default	Notes
`PSE_EDGE_BASE_URL`	`https://edge.pse.com.ph`	
`PSE_MARKET_OPEN` / `PSE_MARKET_CLOSE`	`09:30` / `15:00`	Asia/Manila; drives the freeze
`PSE_THROTTLE_RPS` / `PSE_THROTTLE_BURST`	`1.0` / `2`	**Outbound** politeness
`PSE_TIMEOUT_SEC` / `PSE_RETRY_ATTEMPTS`	`20.0` / `3`	
`DATABASE_URL`	unset	Unset ⇒ in-memory + no archive
`PSE_DB_POOL_SIZE` / `PSE_DB_MAX_OVERFLOW`	`5` / `10`	
`PSE_AUTH_REQUIRED`	`false`	Requires `DATABASE_URL`
`PSE_TOKEN_CACHE_TTL`	`60`	**The revocation-latency budget**
`PSE_QUOTA_PER_MIN` / `PSE_QUOTA_PER_DAY`	`60` / `2000`	Per user, **in-process**
`PSE_PUBLIC_URL`	`http://localhost:8000`	**The highest-risk setting — see below**
`PSE_ACCESS_TTL_MIN` / `PSE_REFRESH_TTL_DAYS`	`30` / `30`	
`ZEPTOMAIL_API_KEY`	unset	Unset ⇒ emails logged to console
`PSE_EMAIL_FROM`	`no-reply@localhost`	Must be a **verified** sender domain
`PSE_USAGE_RETENTION_DAYS`	`90`	The privacy page states 90
`PSE_STATEFUL` / `PSE_SSE`	`false` / `false`	See below
`PSE_LOG_JSON` / `PSE_LOG_LEVEL`	`false` / `INFO`	

> **`PSE_PUBLIC_URL` must be the real external HTTPS URL.** It simultaneously drives the WebAuthn `rp_id`, the links in verification emails, the OAuth issuer in discovery documents, and the DNS-rebinding host allowlist. A wrong value breaks passkeys in a way that looks like a browser bug, and enrolled credentials cannot be recovered — only re-enrolled. Verify it after every deploy: `curl -s https://<host>/.well-known/oauth-protected-resource`

Why HTTP is stateless with plain JSON by default

The server is 12 read-only tools over data the freeze holds still. It uses **none** of the features MCP sessions exist to enable — no notifications, no resource subscriptions, no sampling, no elicitation, no progress. Every request is self-contained.

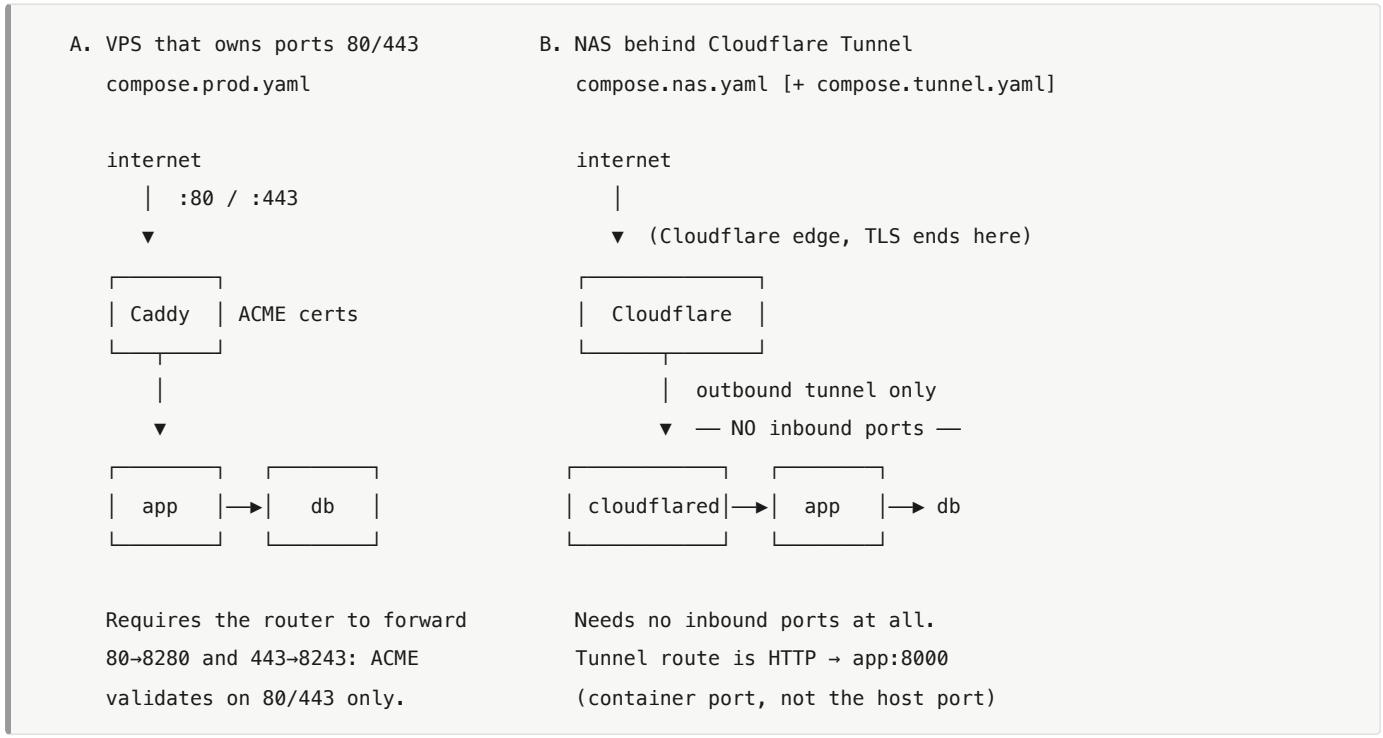
That makes horizontal scaling ordinary: any replica serves any request behind plain round-robin — no sticky routing, no per-session memory, no event store. Without SSE, idle clients hold no connection, so N users stop meaning N concurrent connections.

`--stateful` and `--sse` restore session mode for anyone who needs resumability or server-initiated messages. They are independent flags: statelessness is the scaling property, plain JSON is what keeps ordinary proxies out of the way.

Workers are processes. All in-memory state — quota windows, the parse memo, the token cache, the usage buffer — is **per worker**, so N workers means a user's effective quota ceiling is up to N× nominal. Scale workers for CPU, and scale limits with them.

---

14. Deployment topologies



Both pull the published, CI-gated multi-arch image; neither builds from source. Pin `PSE_IMAGE_TAG` — a moving `:latest` that lags the compose file reads as a broken deployment.

The runtime image contains only what the app needs to run: multi-stage, non-root, no build tools, no dev dependencies, no source tree, no credentials in any layer. `scripts/check_image.py` asserts installed distributions **equal** the resolved runtime closure from `uv export`, so any stray package fails the build. Size is reported as information only — **the gate is necessity, not a number.**

---

15. Gotchas

Findings that cost real debugging time. Most are recorded in `docs/endpoints.md` with the evidence.

- **Parse disclosure tables by `<thead>` label, never by column position.** The two search endpoints order their columns differently.
- **Financial-report units labels contradict each other** between the annual and quarterly sections. Values are passed through **verbatim and never rescaled**; each period reports its own `currency_units` for the caller to check.
- **Index changes are printed unsigned.** PSE Edge shows direction only as a colour and a ▲/▼ glyph, so signs are derived from the glyph.
- **`selectolax` traps:** `iter()` walks direct children only, and `css("a, b")` returns matches **grouped by selector, not in document order** — which once filed all four financial statements under the last heading. Use `root.traverse()`.
- **Accounting negatives** `(1,234)` parse to `-1234`.
- **`dividends_and_rights_form.do` is an empty shell.** It posts to `dividends_and_rights_list.ax` once per tab, with `DividendsOrRights` in the query string and `cmpy_id` in the body.
- **Homepage feeds are keyed by PSE Edge's own group labels**, not invented buckets.
- **`market_calendar.PSE_HOLIDAYS` is a yearly-maintained seed.** Verify it against the PSE holiday circular each December and whenever touching calendar logic.

- `/health` **must never touch the database.** A DB-dependent liveness probe restarts every replica during a blip, turning a recoverable outage into an outage plus a restart storm. `/health/ready` is where the DB check belongs.

---

16. Where to look next

I want to…	Read
Understand why a decision was made	`docs/plan.md`
Add or fix an upstream call	`docs/endpoints.md`, then `client.py` + `parsers.py`
Change what a tool returns	`models.py`, then the repository
Change caching behaviour	`service.py` — and re-read §4 first
Deploy or operate it	`docs/deploy.md`
Know the invariants quickly	`CLAUDE.md`